A Gaussian-Process Emulator for Small-x Gluon TMDs

Setup, tricks, validation, and an honest account of every introduced artifact

Working notes — rcBK/CGC dipole framework (`smallx_tmds`)

July 15, 2026

Abstract

We document, at reproduction level, the Gaussian-process (GP) emulator that maps the two MV-model dipole parameters $\boldsymbol{\theta} = (Q_{s0}^2, C^2)$ to all seven small-x gluon TMDs of Appendix A of arXiv:2512.21466 (plus the Weizsäcker–Williams distribution) on a fixed (k_T, Y) grid — replacing a $\sim$35 s rcBK-plus-Hankel pipeline evaluation by a $\sim$1 ms prediction. The final deployed version (v3) is a *regional* PCA+GP emulator trained on 224 pipeline runs; its held-out accuracy is 0.12% (median relative error of F) with a 95th percentile of 2.5%. We describe the architecture, the design of experiments, and each trick in the order it was introduced, including the negative results (what did *not* help and how that was established). Three deliberate data manipulations are used; each is stated explicitly with its domain of influence and measured cost (Sec. 8).

Contents

1 What is emulated

The target function is the full set of small-x gluon TMDs of the MV dipole model ($\gamma = 1$, $e_c = 1$) evolved with running-coupling BK:

$$\boldsymbol{\theta} = (\ln Q_{s0}^2,\ \ln C^2) \longmapsto \ln F_i(k_T, Y), \qquad i \in \{qg1,\ qg2,\ gg1,\ gg3,\ \text{adj},\ \text{ww}\}, \tag{1}$$

on the fixed grid

k_T	80 points, log-spaced in $[10^{-2}, 10^2]$ GeV
Y	51 slices, $Y = 0, 0.2, \ldots, 10$ $\quad (Y = \ln(x_0/x),\ x_0 = 0.01)$
output dimension	$51 \times 6 \times 80 = 24{,}480$

$\mathcal{F}_{gg}^{(2)} = \mathcal{F}_{gg}^{(1)} - \mathcal{F}_{\text{Adj}}$ is reconstructed exactly from the emulated components and is not emulated separately. The parameter box (= validity region; *no extrapolation outside it*) is

$$Q_{s0}^2 \in [0.05,\ 1.0]\ \text{GeV}^2, \qquad C^2 \in [0.5,\ 30], \tag{2}$$

sampled uniformly in the logs of both parameters.

Why log space. The TMDs span 5–6 decades over the grid and, for the MV model ($\gamma = 1$), are strictly positive everywhere (no near-x_0 negative-TMD artifact, which exists only for $\gamma > 1$). Emulating $\ln F$ therefore loses nothing and makes the outputs $O(1)$ across the whole grid. For an MVγ extension the $Y \lesssim 1$ slices would need signed handling.

The underlying pipeline (the "truth"). One training evaluation runs, per parameter point (`run_pipeline` in `scripts/build_mv_tmd_emulator.py`):

1. rcBK *S-matrix* evolution to $Y = 10$ (`solve_dipole(..., log_gamma=True)`: $N_u = 801$, $u \in [-14, 6]$, DOP853, `NY`=101), which preserves the deep-IR tail ($\Gamma = -\ln S$ exact to $\Gamma \sim 690$) and includes the small-r dilute-limit guard that makes steep configurations solvable;

2. per Y slice, `compute_all_tmds`: since **v6**, the *analytic-model pipeline* – fit the effective anomalous dimension $\varphi = d\ln\Gamma/d\ln r = 2\gamma_{\text{eff}}$ once per slice and build every integrand from analytic derivatives of the fit. No smoothing, no masks, and no A1–A6 tail splices (the shared smooth Γ makes the curves merge into the common perturbative tail by themselves); the one splice kept is $\text{WW} \to \mathcal{F}_{qg}^{(2)}$, an exact physical identity. (History: v5 used the φ-fit for WW with IBP-on-data A1–A6 + adaptive splices; v4 and earlier used the K-fit WW.) The analytic targets are markedly smoother in the deep bands, which is directly visible in the emulator accuracy below;

3. ln of each curve, clipped below at 10^{-12} (generation-level floor), flattened in (Y, i, k_T) order (C-order).

All numerical choices of the pipeline itself (Hankel quadrature, ripple handling, the φ-fit WW) are documented separately in `docs/TMD_numerics.pdf`; the pipeline's own accuracy against analytic references is 0.1–2%, which sets the *useful accuracy ceiling* of any emulator of it. One evaluation costs ≈ 35 s (BK solve $\approx$ 10–15 s, 51×7 Hankel transforms ≈ 25 s).

2 Emulator architecture

2.1 PCA + one GP per principal component (`ShapeEmulator`)

Let $v_n \in \mathbb{R}^{24480}$ be the (conditioned, Sec. 4) $\ln F$ vectors of the N training runs. We center on the mean $\bar{v}$, take the SVD of the residuals, and keep the leading k principal components (PCs) with cumulative explained variance $\geq 1 - 10^{-5}$, capped at $k \leq 36$ (the cap binds; see Sec. 5). Each PC score $z_j(\boldsymbol{\theta})$ is regressed by an independent GP:

$$\ln F(\boldsymbol{\theta}) \approx \bar{v} + \sum_{j=1}^{k} z_j(\boldsymbol{\theta})\, \phi_j, \tag{3}$$

with ϕ_j the PC directions.

2.2 The GP

Anisotropic squared-exponential kernel with a learned noise term,

$$k(\boldsymbol{\theta}, \boldsymbol{\theta}') = \sigma_f^2 \exp\Big[-\tfrac{1}{2} \textstyle\sum_d (\theta_d - \theta'_d)^2 / \ell_d^2 \Big] + \sigma_n^2\, \delta_{\boldsymbol{\theta}\boldsymbol{\theta}'}, \tag{4}$$

inputs and scores standardized to zero mean/unit variance. Hyperparameters $(\ln \ell_d, \ln \sigma_f, \ln \sigma_n)$ maximize the log marginal likelihood (L-BFGS-B, bounds $\ln \ell_d \in [-3, 3]$, $\ln \sigma_f \in [-3, 2]$, $\ln \sigma_n \in [-6, 0.5]$; 3–5 random restarts, first start at $(0, \ldots, 0, -2)$; jitter 10^{-8}). Implementation: `src/emulator.py` (`GP`, `ShapeEmulator`) — pure NumPy/SciPy, ∼150 lines, no external ML dependency.

2.3 Regional composition (`RegionalEmulator`) — the key trick

A single stationary GP over the box (2) floors at ∼1% median error (Sec. 5): the map varies much faster with $\boldsymbol{\theta}$ at low C^2 (large coupling ⇒ fast evolution) than elsewhere — textbook nonstationarity. The fix is two independent `ShapeEmulator`s with *overlapping training ranges*,

region	training range	test/prediction range
low	$C^2 \leq 5.0$	$C^2 < 4$
high	$C^2 \geq 3.2$	$C^2 \geq 4$

blended linearly in $\ln C^2$ over ± 0.15 around the edge $\ln C^2 = \ln 4$, so the combined prediction is continuous (a parameter sweep across the edge shows no visible discontinuity). Predictions inside the blend band are convex combinations of two emulators that were *both trained on that overlap region*.

3 Design of experiments (all seeds and counts)

All designs are Latin-hypercube (`scipy.stats.qmc.LatinHypercube`) in $(\ln Q_{s0}^2, \ln C^2)$. The final training set accumulated in three stages:

stage	points	sub-box	seed
initial train	96	full box	7
initial test	16	full box	1234
augmentation (low-C^2 strip)	64	$C^2 \in [0.5, 4]$	21
augmentation (global)	32	full box	22
regional train (low)	16	$C^2 \in [0.5, 5]$	31
regional train (high)	16	$C^2 \in [3.2, 30]$	32
regional test (low)	8	$C^2 \in [0.5, 5]$	33
regional test (high)	8	$C^2 \in [3.2, 30]$	34
total	**224 train + 32 test**		

Runs are parallelized over 6 worker processes ($\approx$ 6–14 min per batch on a 12-core machine); all raw $\ln F$ vectors are cached (`emu_out/mv_tmd_design.npz`, `regional_extra.npz`), so every retraining below is cache-only. Test points are never used in training, including PCA.

4 Target conditioning — deliberate artifact #1 and #2

Two modifications are applied to the *training targets* (not to the pipeline itself) before PCA+GP. Both act only far below each curve's peak; both are reproducible from the cached raw vectors.

(A1) Per-curve relative floor (per-TMD since v3.2). For each (Y, i) curve with peak $\ln F^{\max}$:

$$\ln F \to \max\left(\ln F,\ \ln F^{\max} - d_i \ln 10\right), \qquad d_{\rm ww} = 8, \quad d_{i\neq \rm ww} = 5.5. \tag{5}$$

The WW floor was deepened after verifying its deep tail is clean across the box (it is the only TMD whose high-k_T tail is spliced onto the exact IBP $\mathcal{F}_{qg}^{(2)}$): WW deep-band held-out median 1.9%, and at the baseline probe the Y=0 WW tail matches the direct pipeline to 0.1% out to $k_T = 100$ GeV. The deep tails of the other TMDs were *measured* unlearnable ($O(1)$ held-out errors) and remain floored at 5.5 decades. *Why:* at low Y and $k_T \gtrsim$ 20–100 GeV several TMDs pass through $F \sim 10^{-7}$–10^{-9} where the fixed-grid Hankel quadrature leaves $O(1)$-*relative* jitter, including negative excursions that the generation-level 10^{-12} clip turns into huge spikes of $\ln F$. These features are deterministic per run (pipeline reproducibility was measured at 10^{-9}) but vary erratically with $\boldsymbol{\theta}$, i.e. they are unlearnable noise that pollutes the PCA basis. *Effect on users:* emulated values more than 5.5 decades below a curve's peak are not faithful (they are flat); everything above is untouched.

(A2) Envelope smoothing of the tiny-F band. Where $F < 10^{-3} F^{\max}$ (per curve), the floored $\ln F$ is replaced by its Savitzky–Golay smoothing (window 9, order 3, along k_T). *Why:* the same relative jitter extends up to $\sim$3 decades below peak; smoothing there removes unlearnable texture while the physical curve (verified smooth in the direct method, Fig. 1) is unaffected within the pipeline's own accuracy. *Effect on users:* fine structure smaller than the SG window ($\sim$0.5 decade in k_T) is not representable below 10^{-3} of a curve's peak. There is no such structure in the physical curves.

Measured gain of (A1)+(A2): held-out median $9.4 \times 10^{-3} \to 4.5 \times 10^{-3}$, 95th percentile $9.9 \times 10^{-2} \to 5.3 \times 10^{-2}$ (global emulator, before regionalization).

5 Development ladder: what worked, what did not

step	held-out median	95th pct
v2: raw targets, single GP, 96 train	9.4×10^{-3}	9.9×10^{-2}
v2.1: + floor (A1)	5.9×10^{-3}	7.2×10^{-2}
+ envelope (A2), 192 train	4.5×10^{-3}	5.3×10^{-2}
+ input warping (drift coordinate)	4.1×10^{-3}	4.8×10^{-2}
v3: + regional split (Sec. 2.3), 224 train	1.2×10^{-3}	2.5×10^{-2}
v4/v4b: spliced-pipeline design (universal adaptive tail splice), per-TMD floors	1.5×10^{-3}	3.5×10^{-2}
v5: WW targets by the unified φ-fit	1.6×10^{-3}	3.5×10^{-2}
v6: ALL targets by the analytic-model pipeline	$\mathbf{8.4 \times 10^{-4}}$	$\mathbf{1.1 \times 10^{-2}}$

v3 per region: high-C^2 5.2×10^{-4} median (the level of the pipeline's own accuracy), low-C^2 3.5×10^{-3}.

v6 (2026-07-15): analytic-model pipeline targets. The full design was regenerated with the analytic-model pipeline (all seven TMDs from the fitted φ; no A1–A6 splices) on the same θ points. Held-out accuracy improved to 8.4×10^{-4} median / 1.1×10^{-2} 95th — a near-2× (median) and 3× (tail) gain over v5, because the analytic targets carry none of the splice kinks and deep-UV quadrature raggedness of the data route (the high-C^2 region now needs only 10 PCs). Emulator-level agreement with v5 is $\leq 0.23\%$ median across the box, with the tails of the difference located exactly where the old targets were noisy. Comparison: `validation/tmds_pipeline_comparison.py`.

v5 (2026-07-15): φ-fit WW targets. The WW channel of the design was regenerated with the unified φ-fit — the production WW default of the pipeline, validated to $\leq 0.8\%$ against the exact analytic $Y = 0$ answers in all four dipole regimes — and the v4b architecture was retrained unchanged. Held-out accuracy is statistically identical to v4b (1.6 vs 1.5×10^{-3} median), and the two emulators agree to median $\leq 0.1\%$ (95th $\leq 1.8\%$) across a dense θ grid: the app's physics is method-independent at the few-percent level. The φ-fit design additionally repaired 74 collapsed ww grid values that the K-fit produced at box corners (656 affected grid values on 21 design points), and broke none. Full confrontation: `emu_out/PHIFIT_REPORT.md`.

Falsified hypotheses (each tested directly, kept for the record):

- *More principal components*: error flat from 24 PCs to full rank (180) — the PCA truncation is not the floor.
- *More training points alone*: learning curve flattens 96→192 at ∼0.9% for the single global GP.
- *Noisy training data*: pipeline re-run with different ODE tolerances/step counts reproduces to 10^{-9} — the data is clean; the jitter of Sec. 4 is deterministic but $\boldsymbol{\theta}$-erratic.
- *Three regions*: no gain over two (1.17 vs 1.23×10^{-3}, worse tails) — low-C^2 error is intrinsic sharpness, not density.

- *Geometric-scaling alignment* (emulate on $k_T/Q_s(Y)$ with a separately emulated scale field): median unchanged, tails *worse* (scale-prediction errors displace the steep shoulder).
- *Y-block emulators* (dedicated PCA+GP for the $Y \leq 2.2$ slices, stitched with an overlap fade): overall median slightly better but $Y \leq 2$ tails worse; the residual low-Y ringing lives in the GP scores, not in the basis capacity.

6 The low-Y ringing and the display filter — artifact #3

The emulator's only visible defect is a small oscillation ("ringing") of the predicted curves at $Y \lesssim 2$, in the $k_T \sim 0.5$–20 GeV band, at some parameter points. Its origin is structural: the low-Y slices contain the sharpest feature of the whole grid (the initial-condition shoulder), whose position moves with $\boldsymbol{\theta}$; a fixed global PCA basis can only represent a *moving* sharp feature through many oscillatory components, and GP errors in their coefficients leave uncancelled ringing (a Gibbs-like phenomenon). It fades by $Y \gtrsim 3$ as evolution smooths the shoulder.

Crucially, the direct pipeline is *not* wiggly there (Fig. 1): measured roughness (RMS second difference of $\ln F$ along $\ln k_T$, $k_T \in [0.5, 20]$) is 0.003–0.03 for the direct curves versus up to 0.05–0.06 for raw v3 predictions at the worst parameter points. An earlier claim (in the working notes) that the truth itself was wiggly at $Y = 1$–2 was an analysis error — the metric had been contaminated by the invisible floored deep-UV tail — and was retracted after the direct overlay test.

(A3) Display-only Savitzky–Golay filter. Because the truth is verified smooth at grid scale, the interactive app applies SG(w=9, p=3) along $\ln k_T$ to the *displayed* $\ln F$. Measured effect: roughness at the worst points $0.052 \to 0.02$ (= the direct level). Honest cost, measured on held-out data: applying the same filter to predictions *increases* the median error $1.23 \to 1.35 \times 10^{-3}$ and the $Y \leq 2$ tail $7 \to 10\%$, because it also rounds the genuine shoulder at extreme parameters. **Therefore the filter is applied in the app display only and is stated in the app caption; the stored emulator (`mv_tmd_emulator.pkl`) returns raw, unfiltered predictions.** A tail-only (selective) variant preserves accuracy but removes only ∼20% of the ringing, since the ringing sits near the shoulder.

7 Validation

Protocol. 32 held-out LHS points (never in training or PCA); metric $|F_{\rm emu}/F_{\rm direct} - 1|$ over all grid points above the conditioning floor. Median and 95th percentile quoted throughout; the error is of F itself (computed as $|e^{\Delta \ln F} - 1|$).

Acceptance overlays. Independent direct pipeline runs at three probe points, compared to v3 over $Y \in \{0, 1, 2, 5\}$ for qg1/qg2/WW (Fig. 2):

probe point	median |rel|	max
app default $(0.104, 15.06)$	6.4×10^{-4}	2.3×10^{-3}
hard low-C^2 corner $(0.30, 1.5)$	2.4×10^{-3}	7.3×10^{-3}
previous worst held-out $(0.064, 7.4)$	2.7×10^{-3}	6.6×10^{-3}

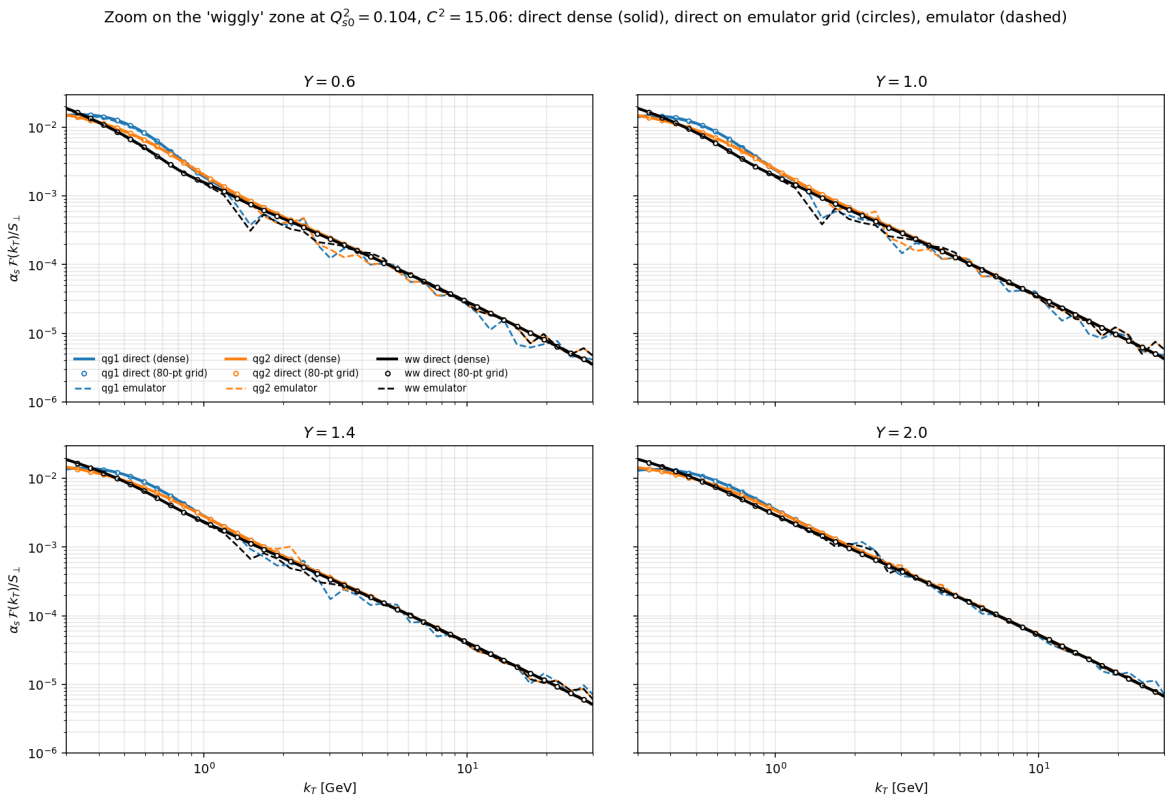

Figure 1: The decisive test (pre-fix, v2-era emulator): direct pipeline computed densely (solid) and on the emulator's own grid (circles) is smooth through the "wiggly" zone; the emulator (dashed) rings. This motivated target conditioning (Sec. 4) and, for the residual, the display filter below.

Known weak spots (quantified). (i) low $C^2 \lesssim 2$: median $\sim 3\times$ worse than the rest; (ii) $Y = 0$ shoulder at extreme parameters: smooth deviations up to $\sim$5–10% of F right at the steepest point; (iii) TMD-crossing bands ($k_T \sim$1–3 GeV, low Y): large *relative* errors on tiny values ($F \sim 10^{-4}$–10^{-5}), oscillating around zero. None of these exceed a few $\times 10^{-3}$ of the local curve scale outside the listed regions.

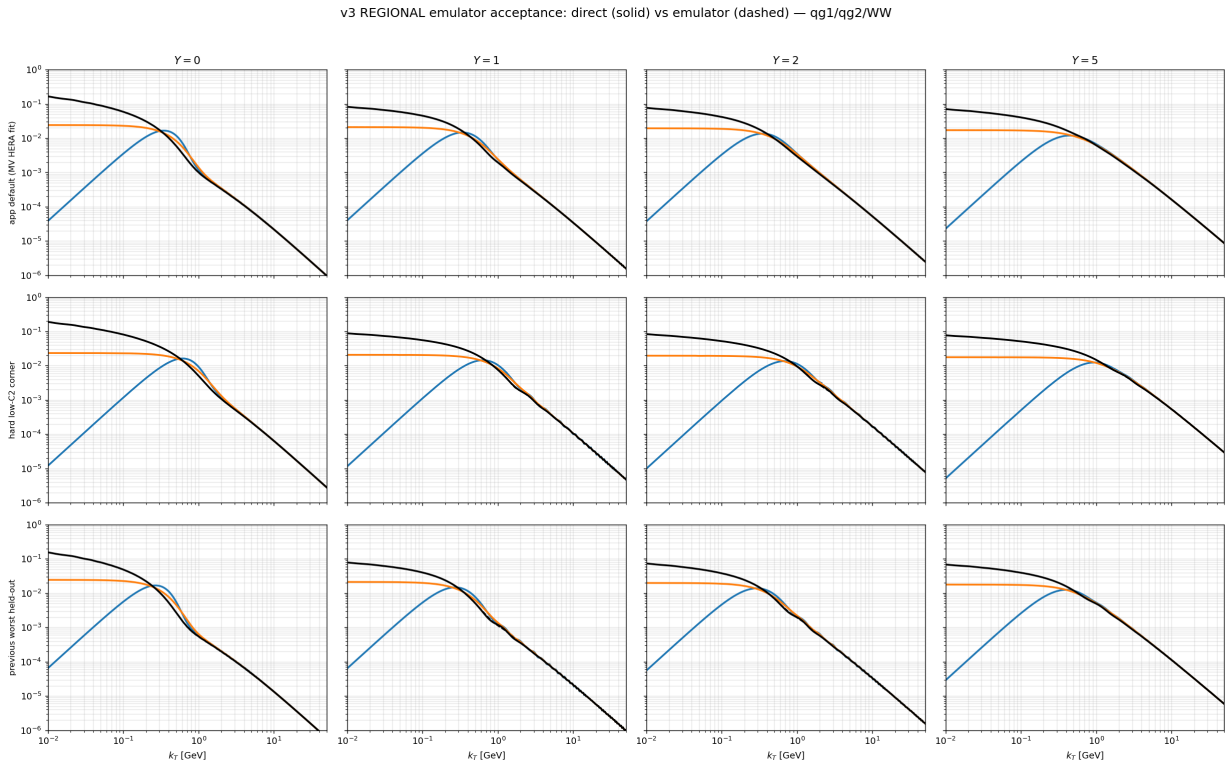

Figure 2: v3 acceptance: direct pipeline (solid) vs regional emulator (dashed), three parameter points × four rapidities, for $\mathcal{F}_{qg}^{(1)}$, $\mathcal{F}_{qg}^{(2)}$, $\mathcal{F}_{\mathrm{WW}}$.

8 Summary of all deliberate artifacts

artifact	where it acts	consequence / cost
(A1) per-TMD relative floor (ww: peak−8 dec; others: peak−5.5 dec)	training targets	emulated F not faithful below the per-TMD floor; the app masks that region
(A2) SG envelope of the $F < 10^{-3}$·peak band	training targets	no sub-window structure representable below 10^{-3} of peak (none exists physically)
(A3) SG(9, 3) display filter	**app display only**, labeled; pickle is raw	visual curves shoulder-rounded by $\lesssim$ the emulator's own error; quantified +10% relative error if it were applied to predictions
blend band $\ln C^2 = \ln 4 \pm 0.15$	predictions	convex combination of two overlap-trained emulators; continuous by construction

No other massaging is applied anywhere: outside these, the emulator is a plain PCA+GP fit of the

pipeline outputs, and the pipeline itself is documented in `TMD_numerics.pdf`.

9 Reproducibility

Code and data (all under `smallx_tmds/`).

- `src/emulator.py` — `GP`, `ShapeEmulator`, `RegionalEmulator`, `ColumnStitchEmulator`, `lhs_design`.
- `scripts/build_mv_tmd_emulator.py` — grid/box constants, `run_pipeline`, initial design (stage-1 seeds above).
- `scripts/regional_design_runs.py` — the regional design batch.
- `scripts/build_mv_tmd_regional.py` — conditioning (A1–A2), regional training, validation, v3 pickle.
- caches: `emu_out/mv_tmd_design.npz` (208 runs, analytic-model pipeline since v6), `emu_out/regional_extra.` (48 runs); the trained emulator: `emu_out/mv_tmd_emulator_v6.pkl`.
- pre-/post-snapshots: `archive/emu_mv_v1`, `emu_mv_v2.1`, `archive/emu_kfit_v4` (K-fit design + v4b pickle), `archive/emu_phifit_v5` (φ-fit-WW design + v5 pickle).

Rebuild from scratch (≈45 min on 12 cores):

```
python3 scripts/build_mv_tmd_emulator.py      # stage-1 design (cached)
python3 scripts/regional_design_runs.py       # regional batch
python3 scripts/build_mv_tmd_regional.py      # condition + train + validate v3
```

Using the emulator.

```
import sys, pickle, numpy as np
sys.path.insert(0, "src")                     # provides emulator.py classes
d = pickle.load(open("emu_out/mv_tmd_emulator_v3.pkl", "rb"))
v = d["emu"].predict_g([[np.log(0.104), np.log(15.06)]])[0]
F = np.exp(v).reshape(len(d["YS"]), len(d["keys"]), len(d["kT"]))
# F[iY, ikey, ikT];  keys = [qg1, qg2, gg1, gg3, adj, ww];  gg2 = gg1 - adj
```

Off-grid (k_T, Y): cubic interpolation of $\ln F$ in $\ln k_T$ and Y (validated to ∼0.2% at worst between Y slices at the v2 stage, better for v3). One evaluation ≈ 0.5 ms.

Environment. macOS ARM, Python 3.9 (system), NumPy/SciPy/Matplotlib; no GPU, no ML libraries. The interactive app (`tmd-explorer/`, Streamlit) bundles the pickle and `emulator.py`.